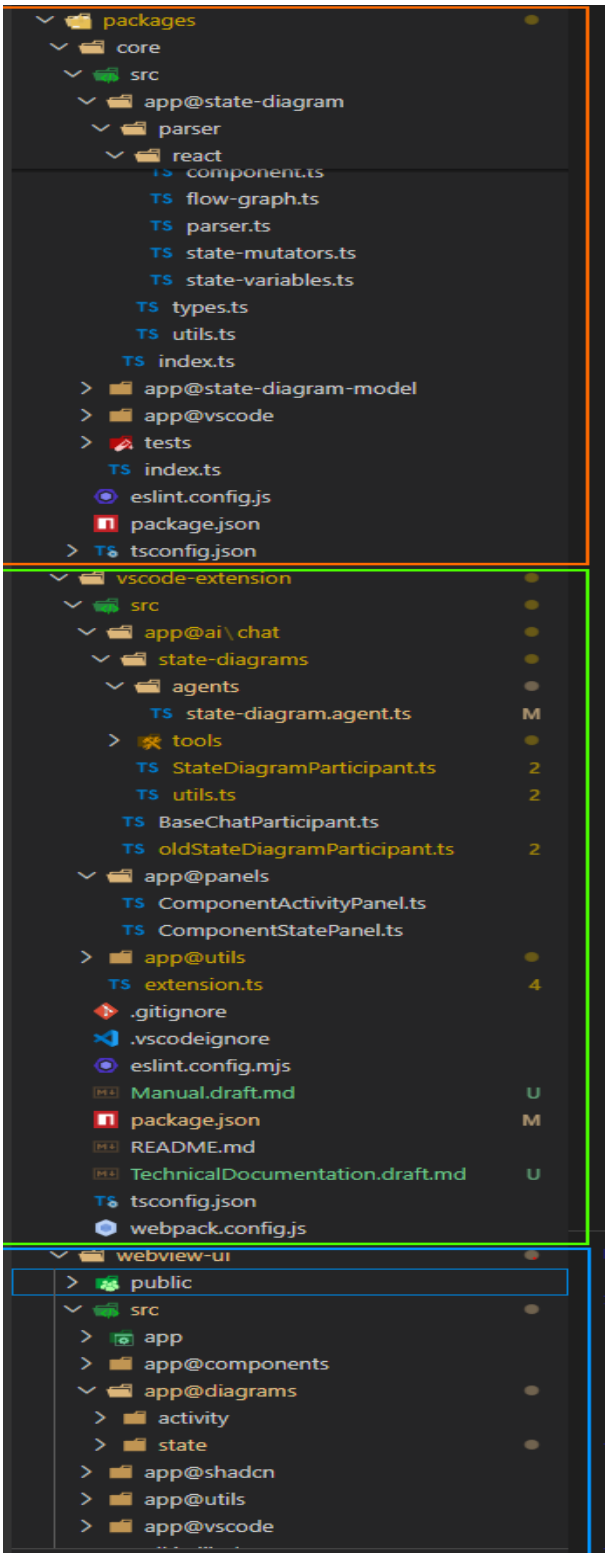
Technical Documentation

Project Structure

The project is organized as a small monorepo with 3 main packages, each owning a separated part of the workflow.



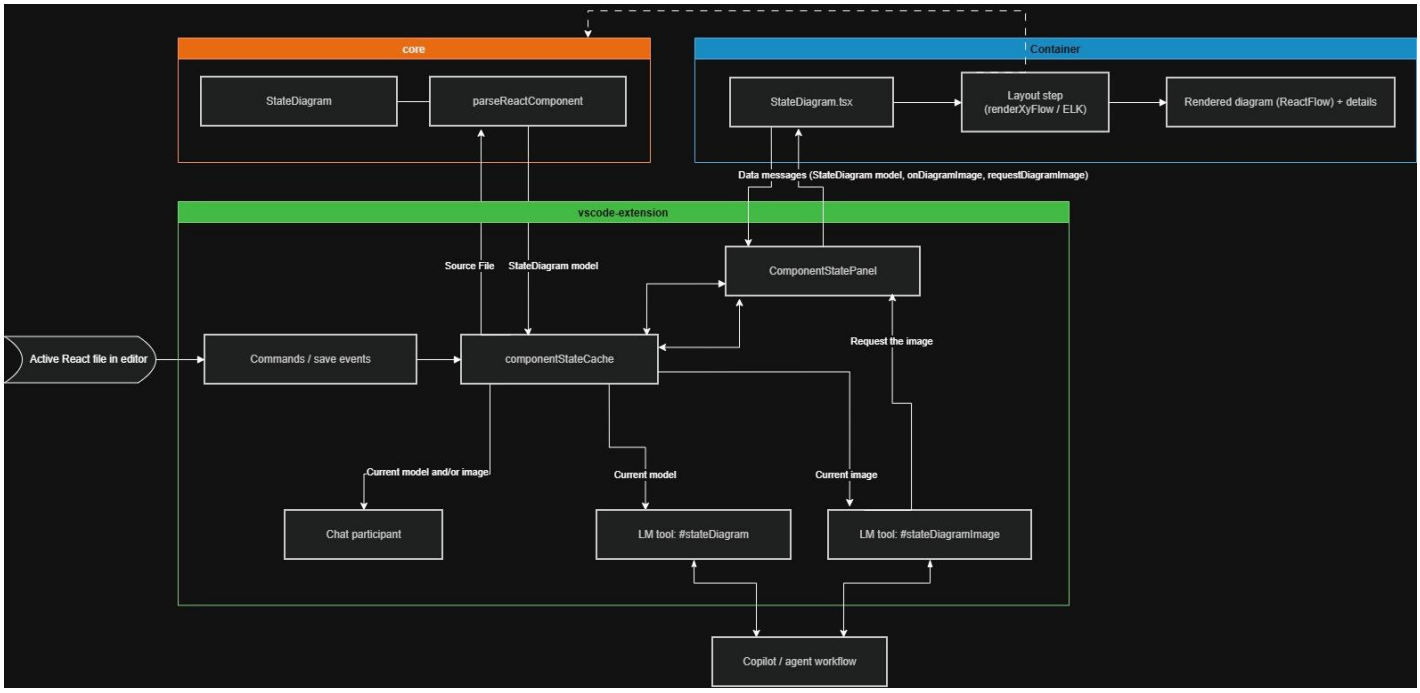
- **core** - Shared parsing and diagram manipulation and building logic. This is where React components are statically analyzed and transformed into the internal State Diagram model representation.
- **vscode-extension** - Extension host code. It registers commands, owns the cache, drives refreshes, exposes AI tools, and coordinates communication between VS Code and the webview.
- **webview-ui** - React-based UI rendered inside the Component State panel. It computes diagram layout and visualizes the result, provides the details panel, generates diagram screenshots on demand etc.

In a web development terms, the **core** can be thought of as Backend, the **vscode-extension** as some sort of Middleware/Executor and **webview-ui** as Frontend.

The extension is implemented with Typescript most prominently utilizing the following technologies:

- **ts-morph** – For parsing the TS/JS representation of the component.
- **React** – The main framework besides the VS Code API.
- **ReactFlow** – For rendering the diagram.
- **Shadcn** – Provide customizable lightweight components.
- **Tailwind** – For styling.
- **VS Code** – As extension host and for interacting with VS Code environment and its features like webview communication, commands, editor interaction, Copilot etc.

High-level data flow



The cache as the single source of truth

componentStateCache is the central integration point of the whole feature.

- It stores the last parsed *StateDiagram* per document.
- It remembers which document is currently considered active/relevant.
- It also stores the most recent diagram image, if one was generated.
- Both the panel and the AI tooling consume data from the same cache, which keeps the diagram representation consistent across the extension.

This means the panel, the participant, and the tools are not separately parsing the file in unrelated ways. They converge on the same cached representation.

Operational notes

- Opening **Component State Panel** triggers parsing through the extension.
- Saving the active document updates cache and refreshes panel view.
- Changing relevant settings clears the cache so diagrams rebuild with new options.
- Layouting is applied after parsing and before final rendering in the webview path (using shared layout helpers).
- Diagram image can be obtained in two ways:
    - **Request Current Diagram Image**: requestDiagramImage -> webview capture -> onDiagramImage -> cache update -> optional disk save.
      The Chat participant or the AI tool requests it in a similar style.
    - **Inside the webview**: User clicks the Screenshot, camera button -> onDiagramImage -> cache update -> optional disk save.

Diagram representation

The central representation is *StateDiagram* (located in the code).

```
export type CodePos = {
  line: number;
  col: number;
};

export type FunctionDeclarationKind = 'function' | 'arrow-function' | 'function-expression';

export type StateHookKind = string /*| 'useState'*/;

export type Id = string /*| number*/;

export type MutationPattern = 'setter-call' | 'ref-current';

export interface StateDiagramComponent {
  name: string;
  pos: CodePos;
  exportName: 'default';
  declarationKind: FunctionDeclarationKind;
}

export interface StateVariable {
```

```
  id: Id;
  hook: StateHookKind;
  name: string;
  setterName: string;
  mutPattern: MutationPattern;
  initializerText?: string;
  typeText?: string;
  pos: CodePos;

  states?: StateUpdate[]; // This will be populated later with the updates related to all the states this variable can reach in all functions that mut it.
  mutators?: StateMutatingFunction[]; // All the functions that mutate this state variable.
}

export interface StateMutatingFunction { // A function that mutates a specific StateVariable.
  id: Id;
  name: string;
  args?: string;
  pos: CodePos;
  type: FunctionDeclarationKind;

  nodes: StateGraphNode[]; // Ordered list of graph nodes (state updates and control flow structure) in this function.
  transitions: StateTransition[]; // Directed edges connecting the graph nodes.
}

export type StateUpdateKind =
  'direct' |
  'expression' |
  'updater';

export type ControlFlowNodeKind =
  'entry' |
  'decision' |
  'try-decision' |
  'switch-decision' |
  'loop-decision' |
  'merge' |
  'exit' |
```

```
    'throw';

export type StateGraphNodeType =
    'state-update' |
    'control-flow';

export type StateGraphNode = StateUpdate | ControlFlowNode;

export interface ControlFlowNode {
    id: Id;
    nodeType: 'control-flow';
    kind: ControlFlowNodeKind;
    label?: string; // txt for decision nodes
    pos: CodePos;
}

export interface StateUpdate {
    id: Id;
    nodeType: 'state-update';
    stateVariableId: Id;
    setterName: string;
    mutPattern: MutationPattern;
    kind: StateUpdateKind;
    pos: CodePos;
    label?: string; // expression txt
}

export enum StateTransitionKind {
    Normal = 'normal',
    Then = 'then',
    Else = 'else',
    Case = 'case',
    Default = 'default',
    Loop = 'loop',
    Catch = 'catch',
    Finally = 'finally',
    Return = 'return',
    Throw = 'throw',
}
```

```
export interface StateTransition {
  id: Id;
  fromNodeId: Id;
  toNodeId: Id;
  kind: StateTransitionKind;
  label: string;
  rawConditionText?: string;
}

export interface StateDiagram {
  component?: StateDiagramComponent;
  stateVariables: StateVariable[];
  source?: string; // Orig component source code or path to source file
}
```

At a high level, the structure is:

Component -> StateVariables -> Mutators -> FlowGraph

More concretely:

- **Component** - Basic metadata about the detected default exported React component.
- **StateVariables** - Each tracked state representation, with its hook kind, name, initializer, mutation pattern, and source position.
- **Mutators** – Functions, arrow functions or scopes that mutate a given state variable.
- **FlowGraph** - Directed graph with nodes and transitions describing how updates happen through control flow. The State flow diagram representation itself.

Part	Purpose
component	Identifies the resolved React component and its declaration metadata
stateVariables	Main payload of the analysis; tracked state-like variables
source	Original input used to build the model

StateVariable

Each state variable records:

- Hook name, such as *useState* or *useRef*
- Variable name and setter name
- Mutation pattern (setter-call or ref-current)
- Initializer/type/source position
- Derived states and mutators discovered later in the pipeline

StateMutatingFunction

Each mutator groups the graph for one function/scope that changes a specific state variable:

- Function identity and source position
- Graph nodes in execution order
- Graph transitions between them

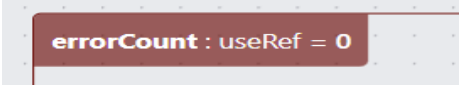
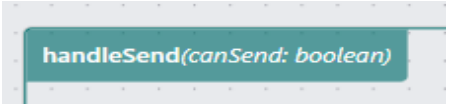
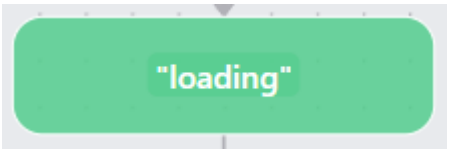
Graph node and transition model

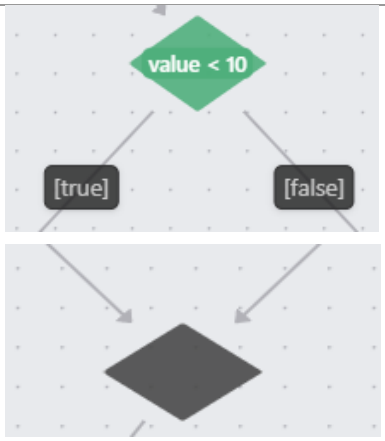
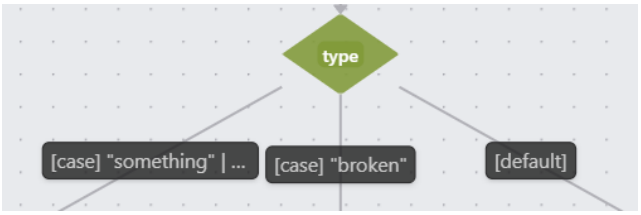
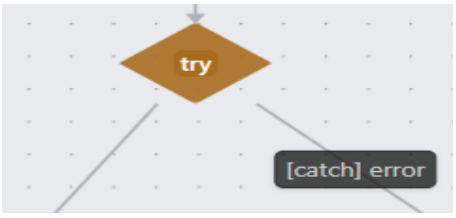
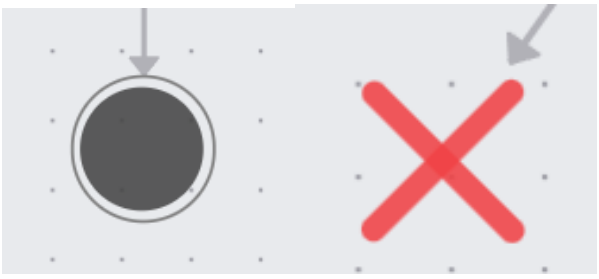
The flow graph combines two node families:

- **State-update nodes** - represent concrete mutations such as setter calls or direct *ref.current* writes
- **Control-flow nodes** - represent branching and structural flow (decision, switch-decision, loop-decision, merge, exit, throw, …)

Transitions then connect those nodes and carry semantic meaning such as then, else, case, catch, finally, return, or throw.

Code-to-diagram mapping

Source code concept	Diagram representation
export default React component	Component (The whole panel view)
useState(...) or configured custom state hook	*StateVariable* with *setter-call* mutation pattern (First group level) 
useRef(...) or configured ref-like hook	*StateVariable* with *ref-current* mutation pattern (First group level) 
function variableMutator(…)	Second group level containing the diagram 
setValue(...)	*state-update* node with state value 
ref.current = ...	*state-update* node with state value 
if / else	decision node + [true]/[false] transitions and subsequent merge, or guard transitions with condition. (Green and gray diamond nodes)

	
switch	*switch-decision* node + [case]/[default] transitions (Grass-green diamond nodes) 
try / catch / finally	try-decision structure + [catch] transition and normal transition for body with (Brown/orange try diamond) 
loops	*loop-decision* node + [loop] loopback dashed transitions (Blue diamond) 
return / throw	exit/throw style nodes (Circle with border, or red X sign) 

State parsing

The parsing pipeline is centered around *parseReactComponent*. It performs static analysis only. No runtime execution, instrumentation, or browser hooks are needed.

Location: **packages/core/src/app@state-diagram/parser/react/parser.ts**.

1. Component resolution

Handled by *resolveDefaultExportComponent* from *app@state-diagram/parser/react/component.ts*.

- The parser receives the current component input and creates a source file representation.
- It tries to resolve the default export of the React component.
- If no valid exported component is found, the result is an empty StateDiagram with no state variables.
- If found, it extracts component metadata needed later for the model.

2. State variable collection

Handled by *collectStateVariables* from *app@state-diagram/parser/react/state-variables.ts*.

- Finds supported state-like declarations.
- Recognizes *useState*, *useRef*, and configured custom hook names.
- Records names, setters, mutation kind, initializers, types, and source positions.

3. Mutator discovery

Handled by *populateStateUpdatesAndMutators* from *app@state-diagram/parser/react/state-mutators.ts*.

- Scans the component for places (functions) where tracked state variables are changed (setter calls…).
- Resolves setter calls and supported ref mutations.
- Groups those updates under mutating functions and adds them to respective variable.

4. Flow graph construction

Handled by *buildTransitionFlowGraph* from *app@state-diagram/parser/react/flow-graph.ts*.

- Walks the relevant ts-morph AST/control-flow structures, utilizing visitor design pattern.
- Turns branches, merges, loops, switches, try/catch blocks, and exits into graph nodes and transitions.
- Connects discovered updates to the surrounding control-flow context.

5. Model assembly

Handled by *createComponentModel* from *app@state-diagram/parser/react/component.ts* and *parseReactComponent* itself.

- The parser assembles and returns the final StateDiagram object containing component info, state variables, mutators, graph nodes, and transitions.
- The temporary source file representation is disposed afterwards.

Pipeline in short

Receive and process source file → resolve component → collect state variables → find mutators and updates → build flow graph → assemble and return *StateDiagram*

Notes on supported patterns

- The implementation is focused on static patterns that can be recognized reliably from source.
- *useState* is the default state-hook model.
- Ref-like tracking is supported through direct *.current* writes on recognized hooks.
- Custom hook names can be configured so third party state helpers fit into the same pipeline.

Notable parts of code

State Diagram Chat Participant – Prompt building

```
override buildLanguageModelMessages(context: StateDiagramChatContext): LanguageModelChatMessage[] {
    const systemInstruction = [
        "Role: expert assistant for TypeScript, TSX, and React and State diagrams.",
        `Answer in: ${this.responseLanguage}.`,
        "Always analyze code together with the state diagram context.",
        "You can explain behavior, suggest refactors, compare code and diagram, and find potential state related issues.",
        ...(context.currentDiagram.status == "available" || context.currentDiagramImage.status == "available" ? [
            "Always state whether and how the diagram influenced your answer.",
            "Prioritize code improvments that would improve or simplify the diagram while keeping original behavior intact.",
        ] : ["If appropriate, inform user that they have to enable participant capabilities and open file with a valid component to get the full answer capabilities."]),
        "Keep answers practical, structured, and implementation-focused, propose concrete code changes.",
    ];

    const effectiveTask = context.userPrompt.length < 5 ? "Provide a short capabilities intro, suggest concrete next prompts, then give best-effort analysis from available context." : context.userPrompt;

    const modelDescription = "Diagram model: " +
        "component = parsed component metadata; stateVariables = component state declarations (outer groups); mutators = functions that mutate respective stateVariable with control-flow and update nodes, (inner groups).";
    const modelDescriptionVisual = "Diagram model: " +
        "component = parsed component metadata; stateVariables = component state declarations (outer groups); mutators = functions that mutate respective stateVariable with control-flow and update nodes, (inner groups).";

    const rootPath = workspace.workspaceFolders?.[0].uri.fsPath;
    const parts: Array<LanguageModelTextPart | LanguageModelDataPart> = [
        new LanguageModelTextPart(`Task request: ${effectiveTask}`),
        new LanguageModelTextPart(`Active file: ${context.currentFilePath || "unavailable"}`),
    ];      You, 5 days ago • State diagram participant refact …

    if (isContextAvailable(context.currentCodeOrSelection)) {
        parts.push(new LanguageModelTextPart("Code:"));
        parts.push(new LanguageModelTextPart(context.currentCodeOrSelection.data));
    } else {
        parts.push(new LanguageModelTextPart(`Code: ${context.currentCodeOrSelection.status}`));
    }

    if (isContextAvailable(context.currentDiagram)) {
        parts.push(new LanguageModelTextPart("State diagram (structured JSON object):"));

        const serializedDiagram = diagramToJson(context.currentDiagram.data, rootPath);
        // console.debug("Serialized diagram:", serializedDiagram);
        parts.push(new LanguageModelTextPart(serializedDiagram));
    } else {
        parts.push(new LanguageModelTextPart(`State diagram: ${context.currentDiagram.status}`));
    }

    if (isContextAvailable(context.currentDiagramImage)) {
        parts.push(new LanguageModelTextPart("State diagram image:"));
        parts.push(LanguageModelDataPart.image(context.currentDiagramImage.data.data, context.currentDiagramImage.data.mimeType.toString()));
    } else {
        parts.push(new LanguageModelTextPart(`State diagram image: ${context.currentDiagramImage.status}`));
    }

    parts.push(new LanguageModelTextPart(isContextAvailable(context.currentDiagramImage) ? modelDescriptionVisual : modelDescription));

    return [
        LanguageModelChatMessage.User(systemInstruction.join("\n")),
        LanguageModelChatMessage.User(parts)
    ];
}
```

GetCurrentStateDiagramTool (#stateDiagram)

```
export default class GetCurrentStateDiagramTool implements LanguageModelTool<{}> {      You, yesterday • State Diagram Agent for agentic workflow + tool…
    2 references | Windsurf: Refactor | Explain | Generate JSDoc | X
    async invoke() {
        console.debug("Invoking GetCurrentStateDiagramTool...");

        const diagram = await getDiagram(doCommonChecksAndGetDoc(componentStateCache.getCurrentDocument())?.targetDocument);

        if (isContextAvailable(diagram)) {
            return new LanguageModelToolResult([
                new LanguageModelTextPart("State diagram (structured JSON object):"),
                new LanguageModelTextPart(diagramToJson({
                    ...diagram.data,
                    hint: "Read 'source' path with 'read' vscode tool to read the corresponding source code if you havent done so, before submitting any code changes."
                }))
            ]);
        }

        return new LanguageModelToolResult([
            new LanguageModelTextPart(`State diagram (structured JSON object): ${diagram.status}`)
        ]);
    }
}
```

GetCurrentStateDiagramImageTool (#stateDiagramImage)

```
export default class GetCurrentStateDiagramImageTool implements LanguageModelTool<{}> {
    2 references | Windsurf: Refactor | Explain | Generate JSDoc | X
    async invoke() {
        console.debug("Invoking GetCurrentStateDiagramImageTool...");

        const entry = await getDiagramImageEntry(doCommonChecksAndGetDoc(componentStateCache.getCurrentDocument())?.targetDocument);

        if (isContextAvailable(entry)) {
            return new LanguageModelToolResult([
                new LanguageModelTextPart("State diagram image:"),
                LanguageModelDataPart.image(entry.data.data, entry.data.mimeType.toString())
            ]);
        }

        return new LanguageModelToolResult([
            new LanguageModelTextPart(`State diagram image: ${entry.status}`)
        ]);
    }
}
```

Handler for messages from Webview

```
private webviewMessageListener(message: any) {
    const { type, data } = message;
    console.debug("Message received from webview:", type, data);

    switch (type) {
        case "refresh":
            void this.refresh(componentStateCache.getCurrentDocument());
            return;
        case "nodeDblClick":
            const uri = componentStateCache?.getCurrentDocument()?.uri;
            if (uri) {
                const pos = data.data.pos;
                // console.debug(pos)
                jumpToPosition(uri, pos.line - 1, pos.col - 1);
            }
            return;

        case "onDiagramImage":
            const { targetDocument } = doCommonChecksAndGetDoc(componentStateCache.getCurrentDocument()) ?? {};
            if (!targetDocument)
                return;

            saveDiagramImage(targetDocument, data, componentStateCache, data?.saveToDisk).then(this.pendingImageRequestResolve).finally(() => {
                this.pendingImageRequestResolve = undefined;
                this.pendingImageRequest = undefined;
            });
            return;
    }
}
```

Parsing cache update function

```
updateEntry(document: TextDocument, parserOptions?, forceUpdate = false) {
    const cacheKey = normalizeFilePath(document.uri.fsPath);
    this.currentDocument = document;

    const cached = this.entries.get(cacheKey);
    if (!forceUpdate && cached && cached.documentVersion == document.version)
    {
        console.debug("Cache hit");
        return cached;
    }

    console.time(`Parsing React component {${cacheKey}}`);
    const data = this.parseAsync(document.uri.fsPath, { ...parserOptions, rootPath: parserOptions?.rootPath ?? getRootPath(document) });
    console.timeEnd(`Parsing React component {${cacheKey}}`);
    // console.trace();

    const entry: CacheEntry<T> = {
        data,
        documentVersion: document.version,
        document,
        updatedAt: Date.now(),
    };
    this.entries.set(cacheKey, entry);
    return entry;
}
```

Visiting relevant parts of the Ts-morph AST during the flow creation

```
if (Node.isIfStatement(what))
    return this.visitIf(what, incoming, hasSetterAhead, context);

if (Node.isTryStatement(what))
    return this.visitTry(what, incoming, hasSetterAhead, context);

if (Node.isSwitchStatement(what))
    return this.visitSwitch(what, incoming, hasSetterAhead, context);

if (Node.isForStatement(what) || Node.isWhileStatement(what) || Node.isForOfStatement(what) || Node.isForInStatement(what))
    return this.visitLoop(what, incoming, hasSetterAhead, context);

if (Node.isDoStatement(what))
    return this.visitDoWhile(what, incoming, hasSetterAhead, context);

if (Node.isLabeledStatement(what)) {
    const label = what.getLabel().getText();
    const statement = what.getStatement();

    if (Node.isForStatement(statement) || Node.isWhileStatement(statement) || Node.isForOfStatement(statement) || Node.isForInStatement(statement))
        return this.visitLoop(statement, incoming, hasSetterAhead, context, label);

    if (Node.isDoStatement(statement))
        return this.visitDoWhile(statement, incoming, hasSetterAhead, context, label);

    if (Node.isSwitchStatement(statement))
        return this.visitSwitch(statement, incoming, hasSetterAhead, context, label);

    return this.visit(statement, incoming, hasSetterAhead, context);
}

if (Node.isBreakStatement(what))
    return this.visitBreak(what, incoming, context);

if (Node.isContinueStatement(what))
    return this.visitContinue(what, incoming, context);

if (Node.isReturnStatement(what))
    return this.visitReturn(what, incoming);

if (Node.isThrowStatement(what))
    return this.visitEnd(what, incoming, 'throw');
```

Source code

Whole source code of the implementation is available online at https://github.com/FIIT-ASICDE/react-diagrams-vscode.

Note however that multiple people were working on this project in the same repository. Most of the state diagram related code was developed on state-diagram branch (https://github.com/FIIT-ASICDE/react-diagrams-vscode/tree/feature/state-diagrams). Branch may no longer exist in far future.